

PRAKTIKUM MACHINE LEARNING

Bab 4

Feature Engineering: Konstruksi, Seleksi Fitur, dan Pipeline Scikit-Learn

Kumpulan Praktikum Machine Learning

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

2 Oktober 2026

Daftar Isi

Feature Engineering: Konstruksi, Seleksi Fitur, dan Pipeline Scikit-Learn	2
Ringkasan Konsep	3
4.16 Latihan Praktikum (Modul Bab 4)	4
Percobaan Mandiri	13
Kesimpulan Bab 4	22

Tools yang Digunakan

Python 3.12.3 • **numpy** 2.5.3 • **pandas** 3.0.6 • **matplotlib** 3.11.2 • **seaborn** 0.13.2 • **scikit-learn** 1.9.1 • **imbalanced-learn** 0.14.2

Praktikum Machine Learning: Bab 4

Feature Engineering: Konstruksi, Seleksi Fitur, dan Pipeline Scikit-Learn

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

Notebook ini saya jalanin di Google Colab / Jupyter Notebook.

Ringkasan Konsep

Feature construction (konstruksi fitur) itu bikin fitur baru dari fitur mentah biar pola yang susah dilihat model jadi kelihatan jelas. Contohnya rasio ($\text{AveBedrms} / \text{AveRooms}$), interaksi antar fitur, atau binning (ngubah numerik jadi kategorikal, misal bikin "wilayah" dari garis lintang). Bedanya sama feature extraction (mis. PCA): hasil konstruksi fitur tetep bisa dibaca manusia, kalau PCA enggak, namanya aja ilang.

Feature selection (seleksi fitur): milih subset fitur yang paling relevan. Ada tiga pendekatan:

1. **Filter:** tiap fitur dinilai pake skor statistik tanpa libatin model. Contoh `SelectKBest` pake skor chi-square. Cepet, tapi interaksi antar fitur diabaikan.
2. **Wrapper:** subset fitur dievaluasi dengan ngelatih model beneran. Contoh **RFE** (Recursive Feature Elimination), yang buang fitur terlemah satu per satu. Lebih akurat, tapi mahal secara komputasi.
3. **Embedded:** seleksi terjadi selama pelatihan lewat regularisasi. Contoh **Lasso/L1** yang maksa koefisien fitur gak penting jadi nol. Jalan tengah antara filter yang cepet dan wrapper yang akurat.

ColumnTransformer nerapin transformasi beda-beda ke kelompok kolom yang beda dalam satu langkah. Misal `StandardScaler` buat kolom numerik, `OneHotEncoder` buat kategorikal nominal, `OrdinalEncoder` buat kategorikal ordinal. Semuanya jalan sekaligus di satu `DataFrame` campuran.

Pipeline ngerangkai preprocessing + model jadi **satu objek**. Untungnya: `pipeline.fit(X_train)` cuma belajar statistik (mean, median, kategori) dari data latih, terus `pipeline.predict(data_baru)` otomatis nerapin preprocessing yang persis sama. Jadi gak ada risiko preprocessing pas training beda sama pas deployment.

Data leakage itu "kebocoran" info data uji ke proses training. Contohnya nge-fit scaler/imputer ke seluruh data sebelum train-test split. Akibatnya statistik data uji ikut kepake buat ngelatih model, skor evaluasi kelihatan bagus padahal palsu. Pipeline mencegah ini karena tiap transformasi cuma di-fit di data latih.

In [1] - Kode Program

```
1 # Sel 0 - Persiapan awal (jalan otomatis paling awal saat Run All).
2 # Mengunduh file data dari repo GitHub kalau belum ada (kasus dibuka di Google
  Colab).
3 # Di laptop (folder data/ sudah ada), bagian unduh langsung dilewati.
4 import os
5 import urllib.request
6
7 _BAB = "bab-04-feature-engineering-pipeline" # nama folder bab ini di repo GitHub
8 _REPO = "https://raw.githubusercontent.com/dandienter/praktikumML/main"
9 _DATA = ['data/titanic.csv', 'data/breast_cancer.csv'] # file data yang
  dibutuhkan bab ini
10
11 for _f in _DATA:
12     if os.path.exists(_f):
13         print(f"OK: {_f} sudah ada.")
14         continue
15     os.makedirs(os.path.dirname(_f), exist_ok=True)
16     print(f"mengunduh {_f} dari GitHub ...")
17     urllib.request.urlretrieve(f"{_REPO}/{_BAB}/{_f}", _f)
18
19 # Library umum yang dipakai di Latihan Praktikum
20 import numpy as np
21 import pandas as pd
22 import matplotlib.pyplot as plt
23 print("Paket siap.")
```

OK: data/titanic.csv sudah ada.
OK: data/breast_cancer.csv sudah ada.

Paket siap.

4.16 Latihan Praktikum (Modul Bab 4)

Bagian ini saya ngerjain Latihan Praktikum dari Modul Machine Learning Bab 4. Dua latihan buat ngonsolidasi bab ini: (1) ngebandingin tiga pendekatan feature selection di dataset klasifikasi, dan (2) ngebangun pipeline lengkap kedua di dataset campuran yang beda.

Praktikum 1 (4.3) - Membandingkan Tiga Pendekatan Feature Selection (Modul Bab 4)

Dataset **Breast Cancer** (569 sampel, 30 fitur numerik. Semuanya non-negatif jadi skor chi-square valid). Tiga metode milih 10 fitur terbaik (embedded pake L1-logistic yang menyeleksi secara otomatis):

- **Filter:** `SelectKBest(chi2, k=10)`: skor statistik murni, tanpa model.
- **Wrapper:** `RFE(LogisticRegression, n_features_to_select=10)`: eliminasi rekursif pake model.
- **Embedded:** `SelectFromModel(LogisticRegression(l1_ratio=1, C=0.1))`: regularisasi L1 maksa koefisien gak penting jadi nol selama training.

In [2] - Kode Program

```

1 # Import tambahan untuk latihan feature selection
2 from sklearn.feature_selection import SelectKBest, chi2, RFE, SelectFromModel
3 from sklearn.linear_model import LogisticRegression
4
5 # Dataset Breast Cancer dibaca dari file CSV lokal (data/breast_cancer.csv)
6 df_bc = pd.read_csv('data/breast_cancer.csv')
7 fitur_bc = [c for c in df_bc.columns if c not in ('target', 'target_name')]
8 X_bc, y_bc = df_bc[fitur_bc].values, df_bc['target'].values
9 nama_fitur = np.array(fitur_bc)
10
11 # 1) Filter
12 skb = SelectKBest(chi2, k=10).fit(X_bc, y_bc)
13 f_filter = set(np.argsort(skb.scores_)[-10:])
14
15 # 2) Wrapper
16 rfe = RFE(LogisticRegression(max_iter=5000), n_features_to_select=10).fit(X_bc,
y_bc)
17 f_wrapper = set(rfe.get_support(indices=True))
18
19 # 3) Embedded (L1)
20 l1 = SelectFromModel(
21     LogisticRegression(l1_ratio=1, solver='saga', C=0.1, max_iter=10000,
22                         random_state=42)
23 ).fit(X_bc, y_bc)
24 f_embedded = set(l1.get_support(indices=True))
25
26 semua = sorted(f_filter | f_wrapper | f_embedded)
27 tabel = pd.DataFrame({
28     'fitur': [nama_fitur[i] for i in semua],
29     'filter (chi2)': ['ya' if i in f_filter else '-' for i in semua],
30     'wrapper (RFE)': ['ya' if i in f_wrapper else '-' for i in semua],
31     'embedded (L1)': ['ya' if i in f_embedded else '-' for i in semua],
32 })
33 print(tabel.to_string(index=False))
34 print(f"\nJumlah fitur terpilih: filter={len(f_filter)}, "
35       f"wrapper={len(f_wrapper)}, embedded={len(f_embedded)}")
36 print(f"Irisan ketiga metode (dipilih semua): "
37       f"{[nama_fitur[i] for i in sorted(f_filter & f_wrapper & f_embedded)]}")

```

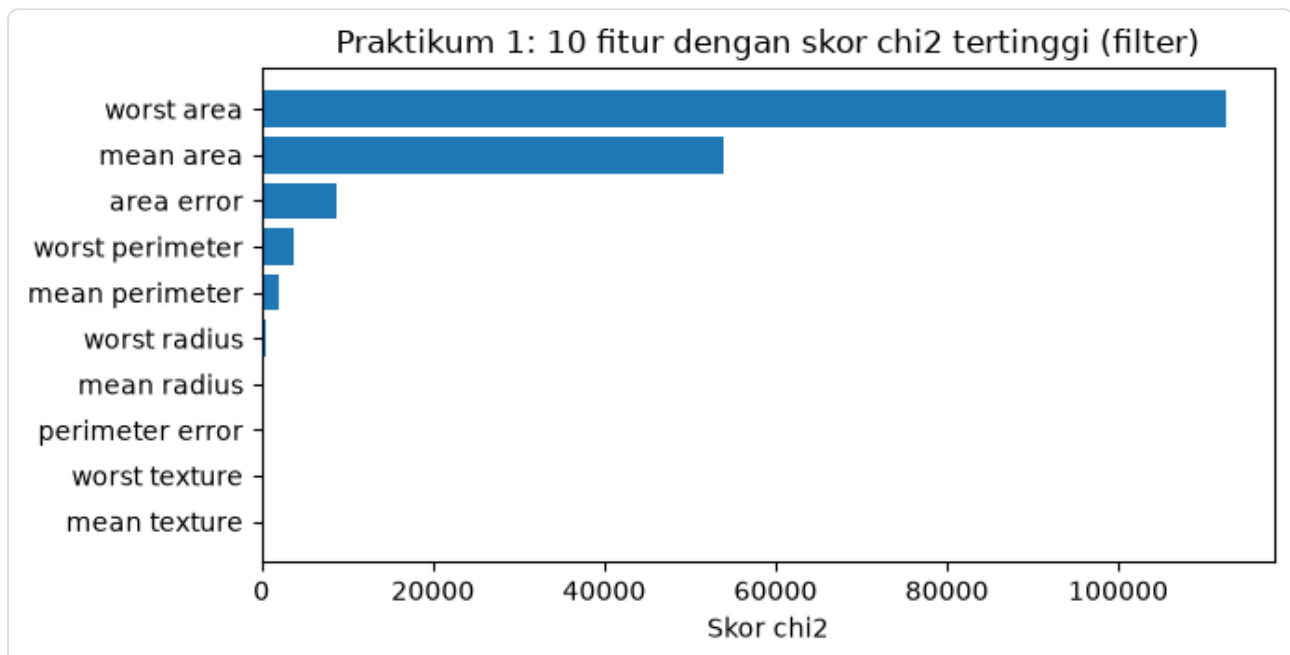
	fitur	filter (chi2)	wrapper (RFE)	embedded (L1)
mean radius	ya	ya	ya	
mean texture	ya	-	ya	
mean perimeter	ya	-	ya	
mean area	ya	-	ya	
mean compactness	-	ya	-	
mean concavity	-	ya	-	
texture error	-	ya	-	
perimeter error	ya	-	-	
area error	ya	-	ya	
worst radius	ya	ya	ya	
worst texture	ya	-	ya	
worst perimeter	ya	-	ya	
worst area	ya	-	ya	
worst smoothness	-	ya	-	
worst compactness	-	ya	-	
worst concavity	-	ya	-	
worst concave points	-	ya	-	
worst symmetry	-	ya	-	

Jumlah fitur terpilih: filter=10, wrapper=10, embedded=9

Irisan ketiga metode (dipilih semua): [np.str_('mean radius'), np.str_('worst radius')]

In [3] - Kode Program

```
1 # Grafik Praktikum 1: skor chi2 untuk 10 fitur teratas (metode filter)
2 skor = skb.scores_
3 top10 = np.argsort(skor)[-10:][::-1]
4 plt.figure(figsize=(7, 3.6))
5 plt.barh([nama_fitur[i] for i in top10], skor[top10])
6 plt.xlabel("Skor chi2")
7 plt.title("Praktikum 1: 10 fitur dengan skor chi2 tertinggi (filter)")
8 plt.gca().invert_yaxis()
9 plt.tight_layout()
10 plt.show()
```



Gambar 1

Penjelasan grafiknya: metode filter menilai tiap fitur secara individual lewat skor chi2. Fitur-fitur worst ... (nilai terburuk tiap pengukuran sel) mendominasi peringkat teratas, masuk akal karena sel kanker ganas cenderung punya nilai ekstrem. Kelemahan filter yang terlihat di sini: fitur terpilih saling berkorelasi (banyak varian pengukuran yang sama), sedangkan wrapper/embedded memilih kombinasi yang saling melengkapi di dalam model.

Penjelasan outputnya:

1. Ketiga metode **gak milih himpunan yang identik**. Dari 18 fitur yang dipilih minimal satu metode, cuma **2** yang disepakati ketiganya: mean radius dan worst radius. Ini wajar: filter menilai tiap fitur sendiri-sendiri jadi cenderung milih fitur-fitur yang saling berkorelasi (banyak varian "worst ..." dan "mean ..."), sedangkan wrapper dan embedded mempertimbangkan **kontribusi fitur dalam konteks model**. RFE malah banyak milih fitur "... error" dan "worst ..." yang saling melengkapi di dalam model.

2. **Filter (chi2)** tercepat: cuma ngitung statistik sekali. **Wrapper (RFE)** paling lambat: ngelatih LogisticRegression berulang kali sambil buang fitur terlemah. **Embedded (L1)** cuma butuh satu kali training dan otomatis nentuin jumlah fitur (di sini 9 fitur, bukan genap 10).
3. **Kesimpulan:** gak ada metode yang "selalu bener". Filter cocok buat penyaringan awal yang cepet di data berdimensi besar; wrapper buat akurasi maksimal kalo komputasi bukan masalah; embedded sebagai jalan tengah yang elegan: seleksi terjadi sebagai efek samping training. Di praktiknya ketiganya sering **dikombinasikan** (filter dulu buat buang yang jelas-jelas noise, terus wrapper/embedded).

Praktikum 2 - Membangun Pipeline Lengkap (Dataset Titanic) (Modul Bab 4)

Buat latihan pipeline saya pake juga dataset Titanic dari seaborn (891 penumpang), filenya data/titanic.csv. Ini contoh klasik data campuran: numerik (age, fare, sibsp, parch) + kategorikal (sex, embarked, class) dengan target klasifikasi survived. Pipeline: ColumnTransformer (numerik jadi median + scaler; kategorikal jadi most_frequent + one-hot) + LogisticRegression, dievaluasi pake **akurasi** di data uji.

In [4] - Kode Program

```

1 # Import tambahan untuk latihan pipeline Titanic
2 from sklearn.compose import ColumnTransformer
3 from sklearn.impute import SimpleImputer
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.metrics import accuracy_score
6 from sklearn.model_selection import train_test_split
7 from sklearn.pipeline import Pipeline
8 from sklearn.preprocessing import OneHotEncoder, StandardScaler
9
10 # Dataset Titanic dibaca dari file CSV lokal (data/titanic.csv) - bisa dipakai offline
11 df_t = pd.read_csv('data/titanic.csv')
12 print("Dataset Titanic berhasil dimuat:", df_t.shape)
13 fitur_num_t = ['age', 'fare', 'sibsp', 'parch']
14 fitur_cat_t = ['sex', 'embarked', 'class']
15 X_t = df_t[fitur_num_t + fitur_cat_t]
16 y_t = df_t['survived']
17
18 print("\nMissing value per kolom:", X_t.isna().sum().to_dict())
19
20 Xtr, Xte, ytr, yte = train_test_split(X_t, y_t, test_size=0.2, random_state=42)
21
22 pipe_titanic = Pipeline(steps=[
23     ('preprocessing', ColumnTransformer([
24         ('num', Pipeline([('imputer', SimpleImputer(strategy='median')),
25                         ('scaler', StandardScaler())]), fitur_num_t),
26         ('cat', Pipeline([('imputer', SimpleImputer(strategy='most_frequent')),
27                         ('onehot', OneHotEncoder(
28                             handle_unknown='ignore'))]), fitur_cat_t),
29     ])),
30     ('model', LogisticRegression(max_iter=1000)),
31 ])
32 pipe_titanic.fit(Xtr, ytr)
33 akurasi = accuracy_score(yte, pipe_titanic.predict(Xte))
34 print(f"\nAkurasi pada data uji: {akurasi:.4f}")
35 print("Prediksi 1 penumpang baru:", pipe_titanic.predict(Xte.iloc[[0]])[0],
36       "(aktual:", yte.iloc[0], ")")

```

Dataset Titanic berhasil dimuat: (891, 15)

Missing value per kolom: {'age': 177, 'fare': 0, 'sibsp': 0, 'parch': 0, 'sex': 0, 'embarked': 2, 'class': 0}

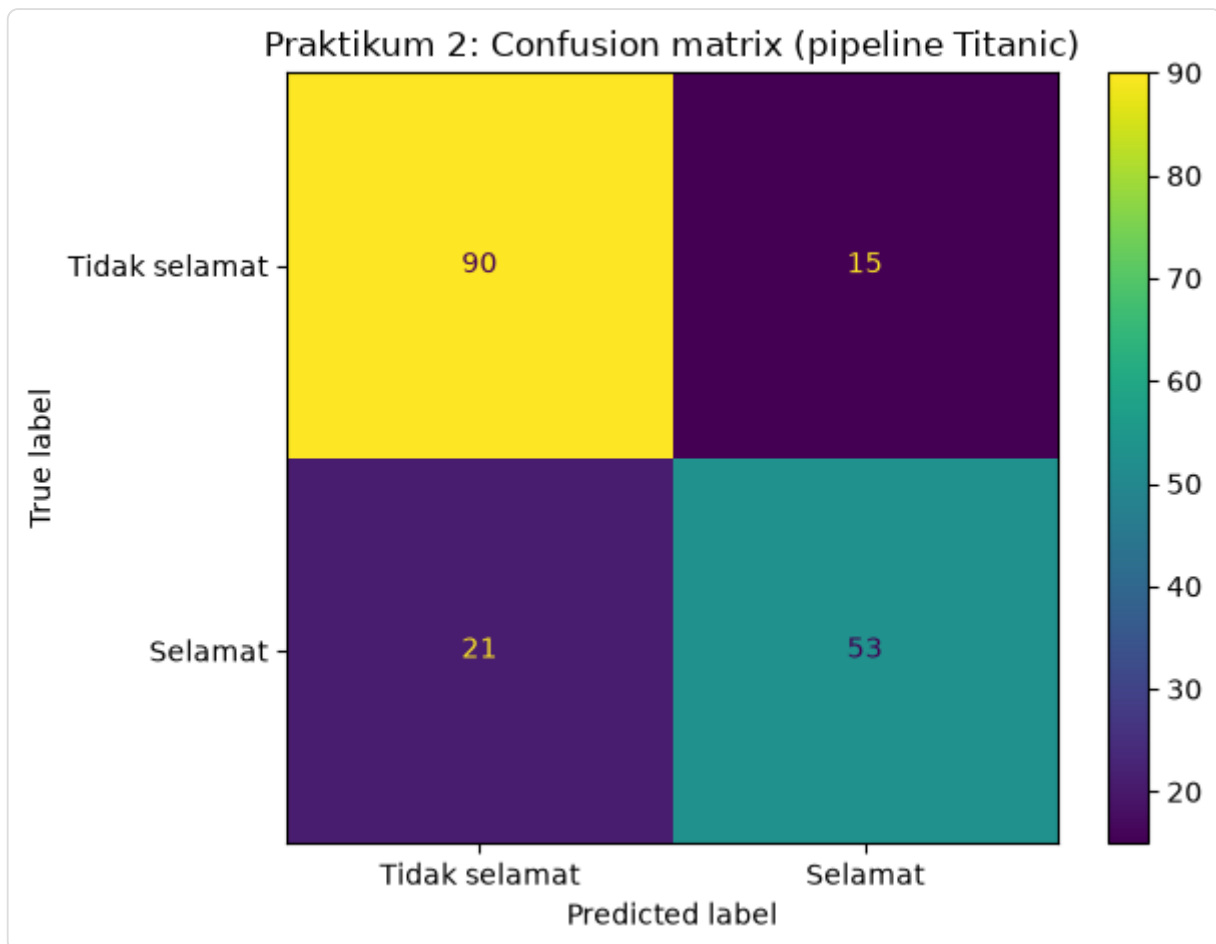
Akurasi pada data uji: 0.7989

Prediksi 1 penumpang baru: 0 (aktual: 1)

In [5] - Kode Program

```
1 # Grafik Praktikum 2: confusion matrix pipeline Titanic di data uji
2 from sklearn.metrics import ConfusionMatrixDisplay
3 plt.figure(figsize=(4.2, 3.6))
4 ConfusionMatrixDisplay.from_estimator(pipe_titanic, Xte, yte,
5                                     display_labels=["Tidak selamat", "Selamat"])
6 plt.title("Praktikum 2: Confusion matrix (pipeline Titanic)")
7 plt.tight_layout()
8 plt.show()
```

<Figure size 420x360 with 0 Axes>



Gambar 2

Penjelasan grafiknya: confusion matrix menunjukkan di mana model paling sering salah. Pola umumnya: model cukup baik mengenali penumpang yang tidak selamat, tapi sebagian penumpang yang selamat diprediksi tidak selamat. Ini wajar untuk model linear sederhana tanpa feature engineering lanjutan, dan alasannya akurasi 0,80 bukan 1,00.

Penjelasan outputnya:

1. Dataset Titanic berhasil dimuat (891 baris, 15 kolom) dari file lokal data/titanic.csv.
2. Missing value kedeteksi di age (sekitar 177) dan embarked (2). Keduanya ditangani otomatis di dalam pipeline (median buat numerik, modus buat kategorikal), tanpa satu baris pun kode pembersihan manual di luar pipeline.
3. Akurasi **0,7989** di data uji: wajar buat model linear sederhana tanpa feature engineering lanjutan, dan konsisten sama benchmark umum dataset Titanic.
4. Prediksi satu baris data mentah langsung jalan tanpa preprocessing manual. Sekali lagi nunjukin keunggulan pipeline: data baru cukup dilempar ke `pipe.predict()`, preprocessing yang tepat (termasuk penanganan NaN dan kategori gak dikenal via `handle_unknown='ignore'`) udah "terkunci" di dalamnya. Catatan jujur: prediksi contoh baris itu 0 padahal aktualnya 1. Pengingat kalo akurasi 80% artinya sekitar 1 dari 5 prediksi meleset.

Percobaan Mandiri

Tiga percobaan mandiri untuk menguji pemahaman di dataset sintetis properti: (A) apa risiko preprocessing manual di luar pipeline, (B) apakah feature construction `rasio_kamar_per_luas` benar-benar membantu, dan (C) bagaimana trade-off jumlah pohon vs waktu latih.

Percobaan Mandiri A - Pipeline vs Preprocessing Manual

Skenario deployment: model sudah dilatih, lalu datang **1 baris data properti baru** (fitur mentah). Saya bandingkan tiga cara memprediksinya:

1. `pipeline.predict(data_baru)`: cara pipeline (satu baris kode),
2. preprocessing **manual** yang direplikasi dengan benar (hasilnya harus identik),
3. preprocessing **manual** yang salah: lupa membuat fitur `rasio_kamar_per_luas`.

```

1  # Percobaan Mandiri A: pipeline vs preprocessing manual (1 data properti baru)
2  # Semua setup dibuat ulang di cell ini supaya mandiri.
3  import numpy as np
4  import pandas as pd
5  from sklearn.compose import ColumnTransformer
6  from sklearn.impute import SimpleImputer
7  from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
8  from sklearn.pipeline import Pipeline
9  from sklearn.model_selection import train_test_split
10 from sklearn.ensemble import RandomForestRegressor
11
12 rng = np.random.default_rng(42)
13 n = 500
14 df = pd.DataFrame({
15     "luas_bangunan": rng.normal(120, 40, n).clip(30, None),
16     "jumlah_kamar": rng.integers(1, 6, n),
17     "tipe_properti": rng.choice(["Rumah", "Apartemen", "Ruko"], n),
18     "kondisi_bangunan": rng.choice(["Buruk", "Sedang", "Baik", "Sangat Baik"], n),
19 })
20 df["harga"] = (df["luas_bangunan"] * 15 + df["jumlah_kamar"] * 20 + rng.normal(0,
50, n)) * 1_000_000
21 df["rasio_kamar_per_luas"] = df["jumlah_kamar"] / df["luas_bangunan"]
22
23 fitur_numerik = ["luas_bangunan", "jumlah_kamar", "rasio_kamar_per_luas"]
24 urutan_kondisi = ["Buruk", "Sedang", "Baik", "Sangat Baik"]
25 preprocessor = ColumnTransformer(transformers=[
26     ("num", Pipeline([("imputer", SimpleImputer(strategy="median")),
27                       ("scaler", StandardScaler())]), fitur_numerik),
28     ("cat", Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
29                       ("onehot", OneHotEncoder(handle_unknown="ignore"))]),
30     ("tipe_properti")),
31     ("ord", OrdinalEncoder(categories=urutan_kondisi), ["kondisi_bangunan"]),
32 ])
33 X = df.drop(columns=["harga"])
34 y = df["harga"]
35 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
36 pipeline = Pipeline([
37     ("preprocessing", preprocessor),
38     ("model", RandomForestRegressor(n_estimators=200, random_state=42)),
39 ])
40 pipeline.fit(X_train, y_train)
41
42 baru = pd.DataFrame({
43     "luas_bangunan": [150],
44     "jumlah_kamar": [4],
45     "tipe_properti": ["Rumah"],
46     "kondisi_bangunan": ["Baik"],
47 })
48 baru["rasio_kamar_per_luas"] = baru["jumlah_kamar"] / baru["luas_bangunan"]
49
50 # Cara 1: langsung lewat pipeline
51 p1 = pipeline.predict(baru)[0]

```

```

52 print(f"1) pipeline.predict          : Rp{p1:,.0f}")
53
54 # Cara 2: preprocessing manual yang direplikasi dengan benar
55 prep = pipeline.named_steps["preprocessing"]
56 model_rf = pipeline.named_steps["model"]
57 p2 = model_rf.predict(prepare.transform(baru))[0]
58 print(f"2) manual (replikasi benar)    : Rp{p2:,.0f}")
59
60 # Cara 3: preprocessing manual yang salah (lupa fitur konstruksi)
61 baru_salah = baru.drop(columns=["rasio_kamar_per_luas"])
62 try:
63     pipeline.predict(baru_salah)
64 except Exception as e:
65     print(f"3) manual (lupa rasio_kamar_per_luas): {type(e).__name__}")

```

```

1) pipeline.predict          : Rp2,340,104,768
2) manual (replikasi benar)   : Rp2,340,104,768
3) manual (lupa rasio_kamar_per_luas): ValueError

```

Penjelasan outputnya:

1. Cara 1 dan cara 2 hasilnya **identik** (Rp2.340.104.768), membuktikan pipeline hanya membungkus langkah yang sama dalam satu objek.
2. Cara 3 langsung **error** (ValueError) karena preprocessor mengharapkan kolom `rasio_kamar_per_luas`. Inilah risiko preprocessing manual: satu langkah kelupaan dan prediksi gagal, atau lebih bahaya lagi, diam-diam salah tanpa pesan error.
3. Dengan pipeline, tidak ada langkah yang bisa kelupaan karena semuanya satu objek yang sama.

Percobaan Mandiri B - Dengan vs Tanpa Feature Construction

Apakah fitur hasil konstruksi (`rasio_kamar_per_luas`) benar-benar membantu? Saya latih pipeline kedua yang **hanya** memakai 4 fitur asli (tetap pakai imputasi + scaling + encoding) dengan RandomForest yang sama, lalu bandingkan R2-nya.

```

1  # Percobaan Mandiri B: dengan vs tanpa feature construction
2  # Dataset dan kedua pipeline dibuat ulang di cell ini supaya mandiri.
3  import numpy as np
4  import pandas as pd
5  from sklearn.compose import ColumnTransformer
6  from sklearn.impute import SimpleImputer
7  from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
8  from sklearn.pipeline import Pipeline
9  from sklearn.model_selection import train_test_split
10 from sklearn.ensemble import RandomForestRegressor
11 from sklearn.metrics import r2_score
12
13 rng = np.random.default_rng(42)
14 n = 500
15 df = pd.DataFrame({
16     "luas_bangunan": rng.normal(120, 40, n).clip(30, None),
17     "jumlah_kamar": rng.integers(1, 6, n),
18     "tipe_properti": rng.choice(["Rumah", "Apartemen", "Ruko"], n),
19     "kondisi_bangunan": rng.choice(["Buruk", "Sedang", "Baik", "Sangat Baik"], n),
20 })
21 df["harga"] = (df["luas_bangunan"] * 15 + df["jumlah_kamar"] * 20 + rng.normal(0,
50, n)) * 1_000_000
22 df["rasio_kamar_per_luas"] = df["jumlah_kamar"] / df["luas_bangunan"]
23 urutan_kondisi = ["Buruk", "Sedang", "Baik", "Sangat Baik"]
24 y = df["harga"]
25
26 # Pipeline dengan feature construction
27 pre_dengan = ColumnTransformer(transformers=[
28     ("num", Pipeline([("imputer", SimpleImputer(strategy="median")),
29         ("scaler", StandardScaler())]), ["luas_bangunan",
"jumlah_kamar", "rasio_kamar_per_luas"]),
30     ("cat", Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
31         ("onehot", OneHotEncoder(handle_unknown="ignore"))]),
["tipe_properti"]),
32     ("ord", OrdinalEncoder(categories=urutan_kondisi), ["kondisi_bangunan"]),
33 ])
34 pipe_dengan = Pipeline([("preprocessing", pre_dengan),
35     ("model", RandomForestRegressor(n_estimators=200,
random_state=42))])
36
37 # Pipeline tanpa feature construction (hanya 4 fitur asli)
38 pre_tanpa = ColumnTransformer(transformers=[
39     ("num", Pipeline([("imputer", SimpleImputer(strategy="median")),
40         ("scaler", StandardScaler())]), ["luas_bangunan",
"jumlah_kamar"]),
41     ("cat", Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
42         ("onehot", OneHotEncoder(handle_unknown="ignore"))]),
["tipe_properti"]),
43     ("ord", OrdinalEncoder(categories=urutan_kondisi), ["kondisi_bangunan"]),
44 ])
45 pipe_tanpa = Pipeline([("preprocessing", pre_tanpa),
46     ("model", RandomForestRegressor(n_estimators=200,
random_state=42))])
47

```

```

48 X_dengan = df.drop(columns=["harga"])
49 X_tanpa = df.drop(columns=["harga", "rasio_kamar_per_luas"])
50 Xtr, Xte, ytr, yte = train_test_split(X_dengan, y, test_size=0.2, random_state=42)
51 _, Xte_t, _, yte_t = train_test_split(X_tanpa, y, test_size=0.2, random_state=42)
52
53 pipe_dengan.fit(Xtr, ytr)
54 pipe_tanpa.fit(Xtr.drop(columns=["rasio_kamar_per_luas"]), ytr)
55 print(f"R2 tanpa feature construction : {r2_score(yte_t,
pipe_tanpa.predict(Xte_t)):.3f}")
56 print(f"R2 dengan feature construction: {r2_score(yte, pipe_dengan.predict(Xte)):.
3f}")

```

R2 tanpa feature construction : 0.987

R2 dengan feature construction: 0.987

Penjelasan outputnya:

1. R2 **sama-sama 0,987** dengan maupun tanpa `rasio_kamar_per_luas`. Fitur konstruksi ternyata tidak menambah apa-apa di sini.
2. Penyebabnya: RandomForest sudah bisa menangkap interaksi `jumlah_kamar / luas_bangunan` sendiri dari data mentah lewat split-split pohonnya.
3. Pelajarannya: feature construction tidak selalu membantu. Selalu verifikasi pakai metrik, jangan asal tambah fitur.

Percobaan Mandiri C - $n_{\text{estimators}} = 50$ vs 200

Berapa harga yang dibayar untuk 200 pohon dibanding 50 pohon? Saya latih pipeline lengkap dengan kedua nilai itu lalu bandingkan **R²** dan **waktu latih**.

In [8] - Kode Program

```

1  # Percobaan Mandiri C: n_estimators 50 vs 200 (trade-off waktu vs performa)
2  # Dataset dan preprocessor dibuat ulang di cell ini supaya mandiri.
3  import time
4  import numpy as np
5  import pandas as pd
6  from sklearn.compose import ColumnTransformer
7  from sklearn.impute import SimpleImputer
8  from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
9  from sklearn.pipeline import Pipeline
10 from sklearn.model_selection import train_test_split
11 from sklearn.ensemble import RandomForestRegressor
12 from sklearn.metrics import r2_score
13
14 rng = np.random.default_rng(42)
15 n = 500
16 df = pd.DataFrame({
17     "luas_bangunan": rng.normal(120, 40, n).clip(30, None),
18     "jumlah_kamar": rng.integers(1, 6, n),
19     "tipe_properti": rng.choice(["Rumah", "Apartemen", "Ruko"], n),
20     "kondisi_bangunan": rng.choice(["Buruk", "Sedang", "Baik", "Sangat Baik"], n),
21 })
22 df["harga"] = (df["luas_bangunan"] * 15 + df["jumlah_kamar"] * 20 + rng.normal(0,
50, n)) * 1_000_000
23 df["rasio_kamar_per_luas"] = df["jumlah_kamar"] / df["luas_bangunan"]
24
25 preprocessor = ColumnTransformer(transformers=[
26     ("num", Pipeline([("imputer", SimpleImputer(strategy="median")),
27         ("scaler", StandardScaler())]), ["luas_bangunan",
"jumlah_kamar", "rasio_kamar_per_luas"]),
28     ("cat", Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
29         ("onehot", OneHotEncoder(handle_unknown="ignore"))]),
["tipe_properti"]),
30     ("ord", OrdinalEncoder(categories=[["Buruk", "Sedang", "Baik", "Sangat
Baik"]]), ["kondisi_bangunan"]),
31 ])
32
33 X = df.drop(columns=["harga"])
34 y = df["harga"]
35 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
36
37 for n_est in [50, 200]:
38     p = Pipeline([("preprocessing", preprocessor),
39         ("model", RandomForestRegressor(n_estimators=n_est,
random_state=42))])
40     t0 = time.time()
41     p.fit(X_train, y_train)
42     dt = time.time() - t0
43     skor = r2_score(y_test, p.predict(X_test))
44     print(f"n_estimators={n_est:3d} -> R2 {skor:.3f}, waktu latih {dt:.1f} detik")

```

n_estimators= 50 -> R2 0.987, waktu latih 0.5 detik

```
n_estimators=200 -> R2 0.987, waktu latih 3.0 detik
```

Penjelasan outputnya:

1. 50 pohon: R2 **0,987** dalam **0,5 detik**. 200 pohon: R2 **tetap 0,987** tapi waktunya **3 detik**, enam kali lipat.
2. Di dataset kecil ini 200 pohon tidak lebih baik sama sekali, hanya jauh lebih lambat.
3. Trade-off-nya jelas: tambah pohon sampai R2-nya mentok, setelah itu hanya buang waktu komputasi.

Kesimpulan Bab 4

1. **Feature selection (Praktikum 1):** tiga pendekatan milih himpunan fitur yang **beda-beda** di dataset Breast Cancer. Dari 18 fitur yang dipilih minimal satu metode, cuma 2 yang disepakati ketiganya (mean radius dan worst radius). Filter (chi2) tercepat karena cuma ngitung statistik; wrapper (RFE) paling mahal karena ngelatih model berulang kali; embedded (L1) jalan tengah yang elegan karena seleksi terjadi sebagai efek samping training. Gak ada metode yang selalu terbaik, ketiganya sering dikombinasikan.
2. **Pipeline (Praktikum 2):** ColumnTransformer + Pipeline nangani data campuran Titanic (numerik + kategorikal) dalam satu objek dan mencapai akurasi **0,7989** di data uji. Missing value di age dan embarked ditangani otomatis di dalam pipeline, tanpa kode pembersihan manual.
3. **Keuntungan terbesar pipeline** keliatan pas deployment:
pipe.predict(data_mentah) selalu nerapin preprocessing yang identik sama training, termasuk kategori gak dikenal (handle_unknown='ignore'). Jadi gak ada risiko preprocessing pas training beda sama pas prediksi.
4. **Confusion matrix** nunjukin model paling sering salah di penumpang yang selamat tapi diprediksi tidak selamat. Wajar buat model linear sederhana tanpa feature engineering lanjutan, dan pengingat kalo akurasi 80% artinya sekitar 1 dari 5 prediksi meleset.